

How to Run the Project

Prerequisites

Before running the project, ensure the following tools are installed on your system:

- Visual Studio Code
- Icarus Verilog
- GTKWave

Setup

1. Create a project folder and place all SystemVerilog source files inside it.
2. Add the top-level design file (`design.sv`) and the testbench file (`testbench.sv`) to the same folder.
3. Open the project folder in Visual Studio Code.
4. Open the integrated terminal in VS code ``

Compile the Design

Run the following command in the terminal:

```
iverilog -g2012 -o sim design.sv testbench.sv
```

This command compiles the processor design and testbench and generates an executable simulation file named `sim`.

Run the Simulation

Execute the compiled simulation:

```
vvp sim
```

During simulation, the testbench generates a waveform file (``dump.vcd``) containing all signal transitions.

View Waveforms

Open the generated VCD file using GTKWave:

```
gtkwave dump.vcd
```

The waveform viewer allows observation of:

- Program Counter (PC)
- Pipeline registers
- ALU outputs
- Register file contents
- Hazard detection signals
- Forwarding paths
- Memory operations

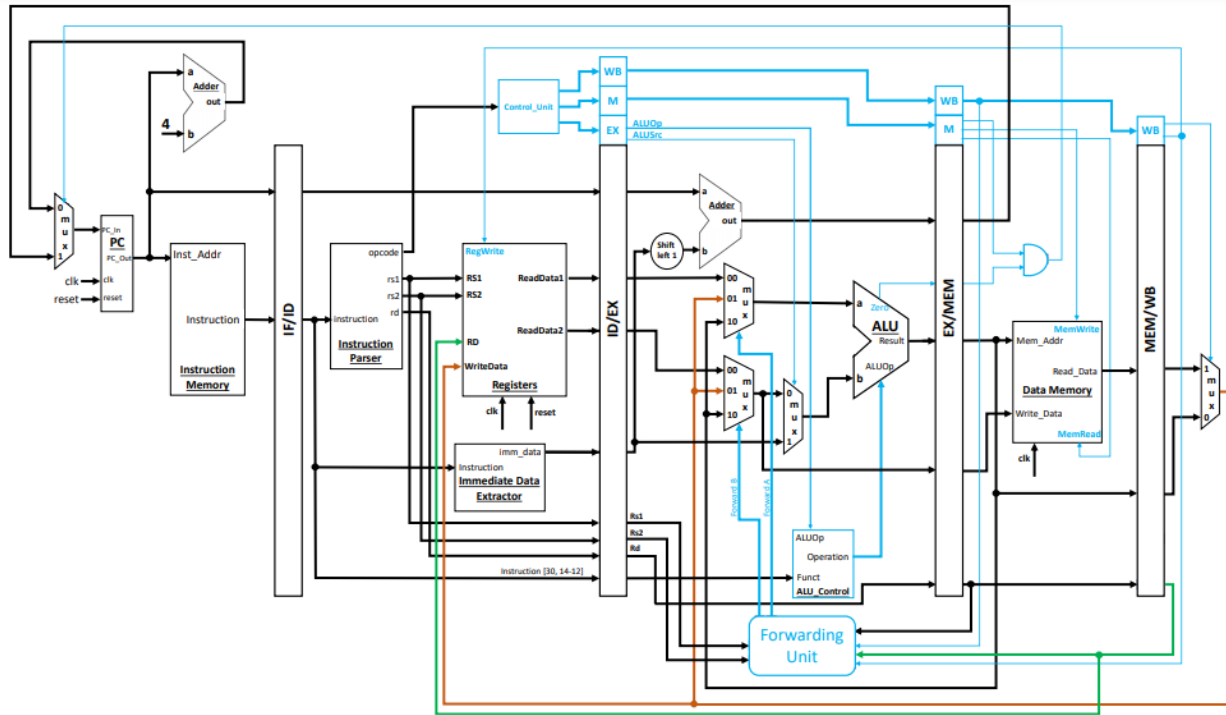
Expected Output

After successful execution:

- The processor fetches, decodes, and executes RISC-V instructions through the five pipeline stages (IF, ID, EX, MEM, WB).
- Hazard detection and forwarding units resolve data dependencies.
- A `dump.vcd` waveform file is generated.

RISC-V Pipelined Processor

This project presents a SystemVerilog implementation of a 32-bit pipelined RISC-V processor. The design incorporates hazard detection, data forwarding, stalling, and pipeline flushing mechanisms to efficiently handle data and control hazards, ensuring correct instruction execution and improved performance. The processor architecture and its major functional units are illustrated in the figure below.



Project Overview

A pipelined processor is a processor architecture that improves performance by dividing instruction execution into multiple stages and allowing several instructions to be processed simultaneously. Instead of completing one instruction before starting the next, the processor overlaps the execution of different instructions across stages such as Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). This increases instruction throughput and makes better use of hardware resources. However, pipelined processors must also handle hazards caused by data dependencies and branch instructions to ensure correct execution.

This project implements a 64-bit RISC-V pipelined processor using SystemVerilog. The design models the complete five-stage pipeline, including instruction fetching, decoding, execution, memory operations, and result write-back. It incorporates essential processor components such as the ALU, register file, instruction memory, data memory, forwarding unit, and hazard detection unit. The processor is simulated using Icarus Verilog, and its operation can be verified through waveform analysis in GTKWave, demonstrating how instructions progress through the pipeline while maintaining correct execution in the presence of hazards.